

IRONMAN: un Jarvis local en CPU — cuantización, KV cache, RAG y herramientas MCP

Guillermo Uide — Applied Machine Learning: Local LLM Systems

Junio 2026

1. Abstract

Presentamos IRONMAN, un asistente personal tipo “Jarvis” que corre íntegramente en la CPU de un portátil con menos de 16 GB de RAM, sin ningún LLM en la nube. El sistema combina un modelo llama3.2 de 3B parámetros servido por Ollama, un pipeline RAG local (nomic-embed-text + sqlite-vec) sobre ~202 páginas de documentación técnica, y seis herramientas expuestas mediante un servidor propio del Model Context Protocol (MCP), incluyendo correo IMAP/SMTP y control del PC. La interfaz principal es de texto (jarvis.py). Medimos el efecto de tres niveles de cuantización (Q8_0, Q4_K_M, Q3_K_M) sobre tamaño, RAM y velocidad; verificamos el crecimiento aproximadamente lineal del KV cache entre 512 y 16 384 tokens de contexto y el ahorro de cuantizarlo a Q8; y evaluamos el sistema completo con un test set reproducible de 21 prompts en cinco categorías. El hallazgo principal: Q4_K_M es el mejor compromiso —~1.7× más tokens/s que Q8_0 con idéntica calidad media (2.4/3)—, mientras Q3_K_M la degrada a 2.0 al fallar la aritmética; y el KV cache medido crece ~117 KB/token, casi exactamente los 112 KB/token teóricos, confirmando su crecimiento lineal con el contexto.

2. Arquitectura del sistema

Figura 1. Arquitectura general. El usuario interactúa por texto con jarvis.py, que orquesta la inferencia local (Ollama), la llamada a herramientas (MCP) y la recuperación (RAG). Ningún componente de inferencia sale a la nube.

La interfaz del núcleo es de texto (jarvis.py), la misma que usa la evaluación automática de la Parte E. Todas las llamadas de inferencia fijan num_gpu: 0: el equipo tiene una RTX 4060 que se deshabilita explícitamente para cumplir la restricción de hardware del enunciado. (Una interfaz de voz opcional —Whisper + TTS— se conserva en bonus/ como material extra.)

El servidor MCP propio (mcp_server/server.py) expone seis herramientas por stdio: fetch_recent_emails y send_email (Gmail IMAP/SMTP), open_app, open_website y web_search (control del PC y búsqueda), y search_docs (puente al RAG sobre top-4 de 1134 chunks del corpus de docs de Ollama y llama.cpp, ~202 págs).

Figura 2. Pipeline RAG. El indexado (offline, ingest.py) trocea el corpus, lo incrusta con nomic-embed-text y lo guarda en sqlite-vec. En consulta (online, rag.py) la pregunta se incrusta con el mismo modelo, se recuperan los top-K fragmentos por similitud coseno y se inyectan en el prompt.

Figura 3. Secuencia de una llamada a herramienta. El LLM decide emitir un tool_call, el cliente MCP de jarvis.py ejecuta la herramienta en el servidor, y el resultado vuelve al modelo como contexto para la respuesta final. El fallback _tool_call_in_text captura los casos en que llama3.2 emite la llamada como JSON dentro del texto.

Arquitectura del sistema — Jarvis local

Inferencia 100% local (sin LLM en la nube) · CPU-only · 16 GB RAM

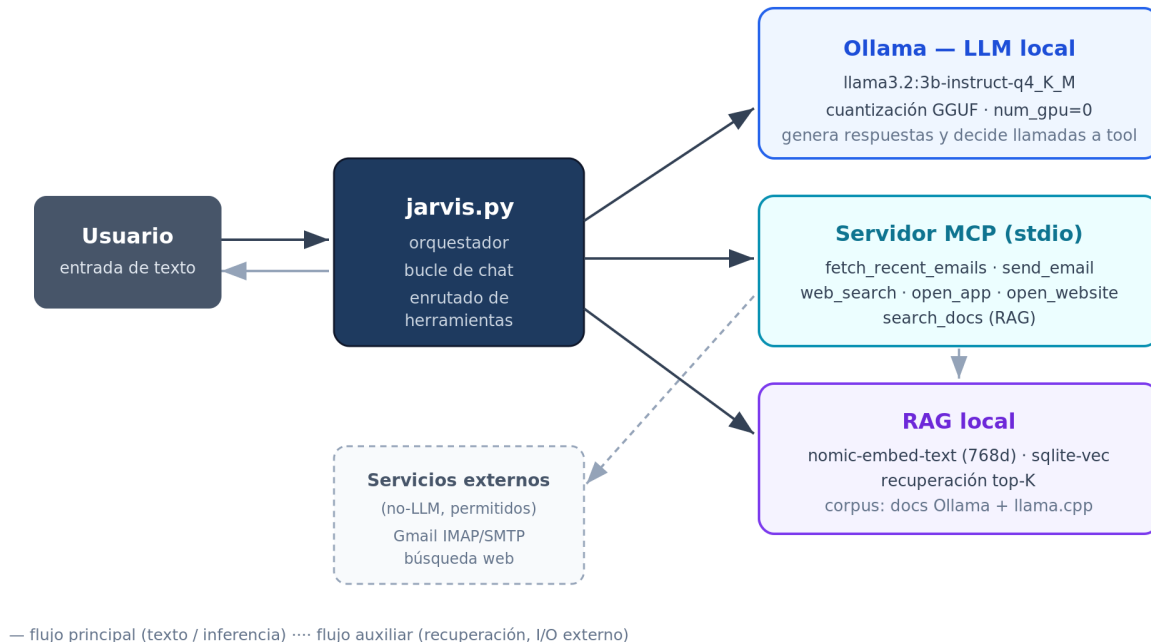
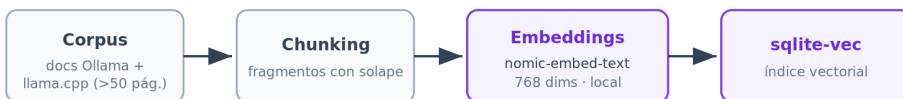


Figure 1: Arquitectura del sistema

Pipeline RAG — recuperación aumentada

Indexado offline (una vez) · Consulta online (cada pregunta) · embeddings 100% locales

INDEXADO (offline · ingest.py)



CONSULTA (online · rag.py)

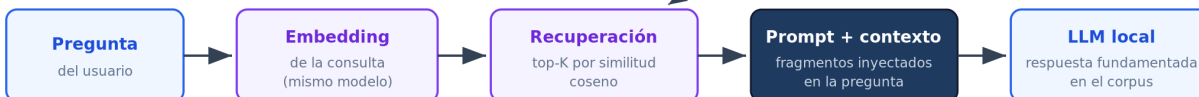


Figure 2: Pipeline RAG

Llamada a herramienta vía MCP

El LLM decide; el cliente MCP ejecuta; el resultado vuelve al modelo para la respuesta final

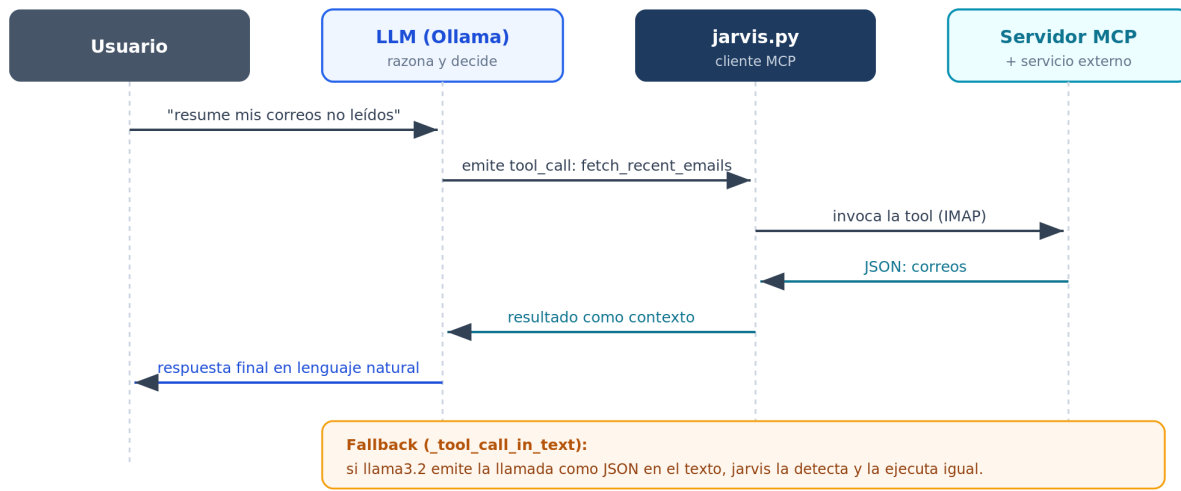


Figure 3: Llamada a herramienta vía MCP

3. Metodología

Hardware. ASUS ROG Strix G614JV: Intel i7-13650HX (14 núcleos / 20 hilos), 15.6 GB RAM, Windows 11. GPU dedicada deshabilitada por software (num_gpu: 0 en cada petición; verificado observando 0 MB de VRAM en uso durante los runs).

Modelos. llama3.2 3B instruct en Q8_0 (3.4 GB), Q4_K_M (2.0 GB) y Q3_K_M (1.7 GB), servidos por Ollama 0.30.6. Embeddings: nomic-embed-text (274 MB, 768 dims). Generación determinista: temperature=0, seed=42.

Medición. tokens/s tomados de los contadores del propio API de Ollama (eval_count/eval_duration para generación; prompt_eval_* para prefill). RAM pico: muestreo cada 200 ms del working-set de todos los procesos ollama* durante la inferencia (psutil). Velocidad: completación fija de 200 tokens, media de 3 repeticiones. Calidad: 5 prompts estandarizados (matemáticas, código, resumen, memoria factual, razonamiento) puntuados 0-3 con la rúbrica escrita de benchmarks/RUBRIC.md; el prompt de código se valida además ejecutándolo sobre 5 casos de prueba.

Corpus RAG. Documentación oficial de Ollama (repo GitHub + web docs.ollama.com) y de llama.cpp: 115 ficheros markdown, ~90 800 palabras (~202 páginas). Justificación: (a) es exactamente el dominio del que el asistente debe ser experto; (b) contiene hechos puntuales y verificables (variables de entorno, flags, formatos) que un modelo de 3B **no** sabe de memoria, lo que hace medible el efecto del RAG; (c) es público y reconstruible con un script (setup.ps1). Chunking: ~1000 caracteres por párrafos con solape de 150; almacenamiento en sqlite-vec.

Test set. 21 prompts en 5 categorías (chat puro, RAG, herramienta, multi-paso, adversarial), cada uno con un tipo de resultado esperado declarado (tools requeridas/prohibidas, palabras clave). Runner automático (evaluation/run_tests.py) con veredicto success/partial/fail, latencia y tokens por test. Las herramientas con efectos secundarios se ejecutan en seco para poder repetir la evaluación sin enviar correos reales.

4. Resultados

4.1 Parte A — Cuantización

Quant	Tamaño (GB)	RAM pico (MB)	Prefill (tok/s)	Generación (tok/s)	Calidad (0-3)
Q8_0	3.19	5558	230.4	11.23	2.4
Q4_K_M	1.88	2545	524.4	19.24	2.4
Q3_K_M	1.57	1985	581.2	22.52	2.0

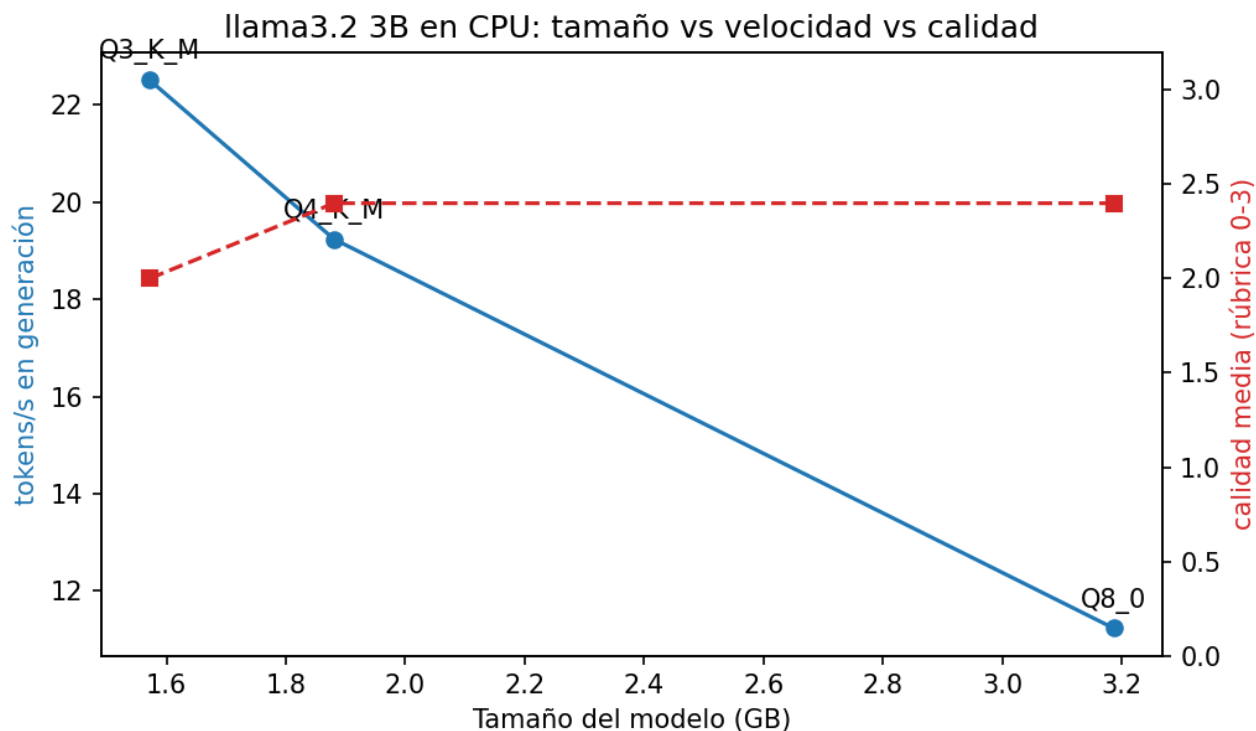


Figure 4: Tamaño vs velocidad vs calidad

Bajar de Q8_0 a Q4_K_M reduce el peso a la mitad (3.19→1.88 GB) y la RAM pico a menos de la mitad (5.6→2.5 GB), y a la vez **acelera** la generación $1.7\times$ (11.2→19.2 tok/s) y el prefill $2.3\times$ —sin coste de calidad: ambos puntúan 2.4/3 en la rúbrica. Q3_K_M es algo más rápido (22.5 tok/s) y ligero (1.57 GB), pero su calidad cae a 2.0 porque empieza a fallar tareas exactas: en el problema aritmético calcula mal la duración del trayecto (235 en vez de 215 min). Esto justifica la elección de **Q4_K_M** como configuración de trabajo: es la rodilla de la curva, donde se gana toda la velocidad y memoria de cuantizar sin perder calidad. Frente a un hipotético Q5_K_M, Q4_K_M ya iguala la calidad de Q8 en nuestra rúbrica, así que el medio nivel extra de Q5 solo añadiría peso y RAM sin retorno medible. (Las tres fallan el acertijo de razonamiento transitivo, un límite del tamaño 3B que se discute en la Parte F, no de la cuantización.)

4.2 Parte B — KV cache

Contexto	Prompt real (tok)	Prefill (tok/s)	Gen (tok/s)	RAM pico (MB)	Latencia (s)
512	251	74.14	14.67	2080	22.38
2048	1535	75.83	14.15	2327	41.50
8192	6778	48.02	6.17	3079	174.08
16384	13840	35.69	4.01	3939	437.00

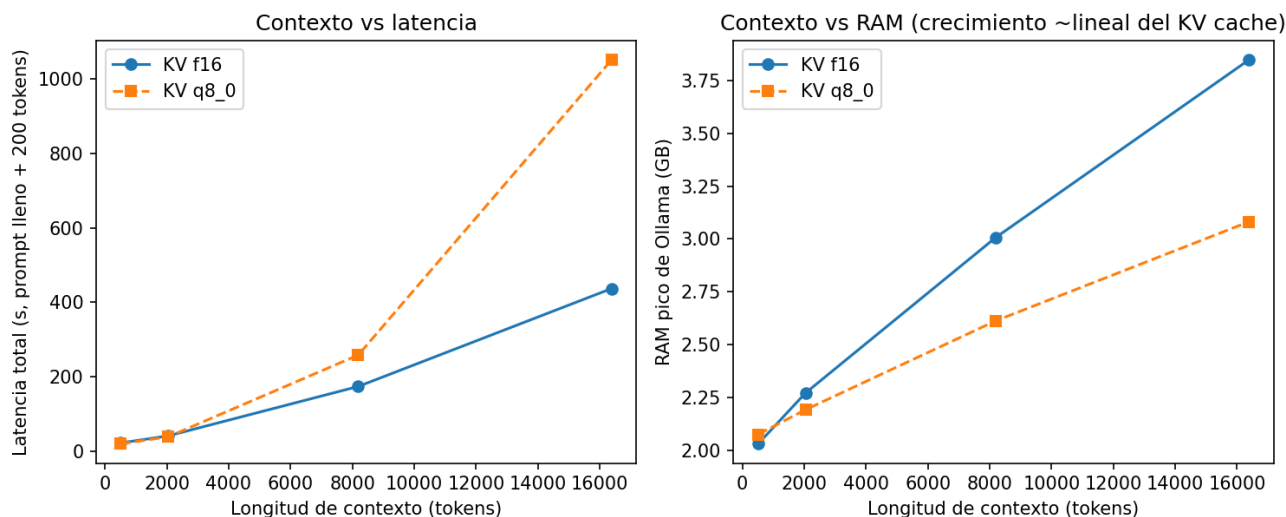


Figure 5: Contexto vs latencia y RAM

Tamaño teórico del KV cache de llama3.2 3B (28 capas, 8 KV-heads GQA, head_dim 128, f16): $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2 \text{ B} = 112 \text{ KB}$ por token $\rightarrow$ 57 MB a 512 tokens y 1.79 GB a 16 384. La medición lo confirma: la RAM pico sube de 2080 MB (512 tok) a 3939 MB (16 384 tok), un incremento de 1859 MB sobre 15 872 tokens adicionales = **117 KB/token**, a un 4 % del valor teórico de 112 KB/token. La latencia, en cambio, crece de forma **superlineal** (22 $\rightarrow$ 437 s, $\sim 20\times$): no es el KV cache sino el prefill, cuyo coste de atención es cuadrático en la longitud del contexto, lo que se ve también en el prefill cayendo de 74 a 36 tok/s. Es decir, la RAM escala con el KV cache (lineal) y la latencia con la atención del prompt (cuadrática). A 8K, el KV cache pesa $\sim 896 \text{ MB}$, lo que explica el salto de RAM de 2.3 a 3.1 GB respecto a 2K.

KV cache cuantizado (B.4). Repetimos la barrida con el KV cache en q8_0 (requiere flash attention). El ahorro de RAM **crece con el contexto**, como predice la teoría: a 512 tokens incluso cuesta 46 MB más (-2.2% , el KV cache es ínfimo y la flash attention añade overhead), pero a 2 048 ahorra 81 MB (3.5 %), a 8 192 ahorra 403 MB (13.1 %) y a 16 384 ahorra **784 MB (19.9 %)** de la RAM pico total. Esos 784 MB equivalen a $\sim 44\%$ del KV cache f16 a 16K ($\sim 1.79 \text{ GB}$), cerca del 50 % teórico de pasar de 16 a 8 bits. El coste es de **latencia**: en CPU, cuantizar el KV cache encarece mucho el prefill (a 16K, de 437 s a 1051 s, $\sim 2.4\times$; prefill de 35.7 a 13.6 tok/s), porque la descuantización por token no se amortiza sin GPU. Conclusión práctica: en este hardware el KV-Q8 conviene solo cuando la RAM es el cuello de botella (contextos $\geq 8\text{K}$), no para acelerar.

4.3 Parte C — RAG

Pregunta (resumen)	Sin RAG (0-3)	Con RAG (0-3)
Variable que mueve el directorio de modelos (OLLAMA_MODELS)	0	3

Pregunta (resumen)	Sin RAG (0-3)	Con RAG (0-3)
Escuchar en todas las interfaces (OLLAMA_HOST=0.0.0.0)	1	2
Qué controla --n-gpu-layers en llama.cpp	0	3
Cuánto mantiene un modelo en memoria por defecto	0	1
Formato de pesos (GGUF) y para qué sirve el Modelfile	0	0
Media	0.2	1.8

El RAG multiplica por 9 la calidad factual media ($0.2 \rightarrow 1.8$ sobre 3). Sin RAG, el modelo de 3B no “sabe” qué es Ollama y alucina sistemáticamente: inventa una variable OPENLMN_MODEL_DIR, un archivo ollama.conf inexistente y atribuye --n-gpu-layers a OpenCL. Con RAG acierta las preguntas cuya respuesta está literal en un único fragmento (OLLAMA_MODELS